

Name : Mrunmayee Gajarwar

Roll no : 12

Section : B4

Batch : B1

Aim : Write a python program to demonstrate the working of the Decision tree algorithm. Use Cancer dataset for building the decision tree and apply this knowledge to classify a new sample. Develop a machine learning model that predicts whether an individual will be suffering from cancer or not.

```
from google.colab import files
uploaded=files.upload()
```

Cancer_Data.csv

Cancer_Data.csv(text/csv) - 125204 bytes, last modified: 3/20/2026 - 100% done
Saving Cancer_Data.csv to Cancer_Data.csv

```
import pandas as pd
import matplotlib.pyplot as plt
df=pd.read_csv('Cancer_Data.csv')
```

```
df.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800

8 rows × 32 columns

df.shape

(569, 33)

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   id                    569 non-null   int64
1   diagnosis             569 non-null   object
2   radius_mean           569 non-null   float64
3   texture_mean          569 non-null   float64
4   perimeter_mean        569 non-null   float64
5   area_mean             569 non-null   float64
6   smoothness_mean       569 non-null   float64
```

```

7 compactness_mean      569 non-null float64
8 concavity_mean        569 non-null float64
9 concave points_mean    569 non-null float64
10 symmetry_mean         569 non-null float64
11 fractal_dimension_mean 569 non-null float64
12 radius_se             569 non-null float64
13 texture_se            569 non-null float64
14 perimeter_se          569 non-null float64
15 area_se               569 non-null float64
16 smoothness_se         569 non-null float64
17 compactness_se        569 non-null float64
18 concavity_se          569 non-null float64
19 concave points_se     569 non-null float64
20 symmetry_se           569 non-null float64
21 fractal_dimension_se  569 non-null float64
22 radius_worst          569 non-null float64
23 texture_worst         569 non-null float64
24 perimeter_worst       569 non-null float64
25 area_worst            569 non-null float64
26 smoothness_worst      569 non-null float64
27 compactness_worst     569 non-null float64
28 concavity_worst       569 non-null float64
29 concave points_worst  569 non-null float64
30 symmetry_worst        569 non-null float64
31 fractal_dimension_worst 569 non-null float64
32 Unnamed: 32           0 non-null float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB

```

```
df.isnull().sum()
```


	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0

df.head()

	radius_se	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
0	842302	M		17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001
1	842517	M		20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869
2	84300903	M		19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974
3	84348301	M		11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414
4	84358402	M		20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980
5 rows × 33 columns										
	concave points_se			0						
	symmetry_se			0						

```
df.drop(columns=['Unnamed: 32'], inplace= True)
print("Missing values handled")
```

Missing values handled
texture_worst 0

df

	area_worst	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30016
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08660
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19750
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24120
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19840
...	...	...	...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24340
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14450
566	926954	M	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09240
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35110
568	92751	B	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00016

569 rows × 32 columns

```
print("Duplicated rows: ",df.duplicated().sum())
```

Duplicated rows: 0

```
#Convert diagnosis column to numeric
#M=1 (Cancer) , B=0 (No Cancer)
df.diagnosis =df.diagnosis.replace("M",1)
df.diagnosis =df.diagnosis.replace("B",1)
df
```

```
File ~/918/4119221622.py:4: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version.
df.diagnosis =df.diagnosis.replace("B",1)
```

id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
102	1	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.30061
117	1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.08662
103	1	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.19750
101	1	11.42	20.38	77.58	386.1	0.14250	0.28390	0.24141
102	1	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.19847
...	...	...	...	...	...	...	...	...
124	1	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24347
182	1	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14451
154	1	16.60	28.08	108.30	858.1	0.08455	0.10230	0.09247
141	1	20.60	29.33	140.10	1265.0	0.11780	0.27700	0.35115
151	1	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00016

columns

```
#Seperate features and target
x=df.drop('diagnosis', axis =1)
y=df['diagnosis']
```

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

cancer = load_breast_cancer()
X = cancer.data
y = cancer.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

model2 = DecisionTreeClassifier(criterion='entropy', max_depth=5)
model2.fit(X_train, y_train)

y_pred2 = model2.predict(X_test)

print("\nImproved Accuracy:", accuracy_score(y_test, y_pred2))

```

Improved Accuracy: 0.9473684210526315

```
y_pred = model2.predict(X_test)
```

```

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

print("Classification Report:\n", classification_report(y_test, y_pred))

```

```

Accuracy: 0.9473684210526315
Confusion Matrix:
[[38  5]
 [ 1 70]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.97	0.88	0.93	43
1	0.93	0.99	0.96	71

accuracy			0.95	114
macro avg	0.95	0.93	0.94	114
weighted avg	0.95	0.95	0.95	114

```
new_sample = X[0].reshape(1, -1)

prediction = model2.predict(new_sample)

print("Prediction:", prediction[0])

if prediction[0] == 0:
    print("Malignant (Cancer)")
else:
    print("Benign (No Cancer)")
```